

**Nadzirano učenje –
klasifikacija –
neparametarski
modeli**



Sadržaj

- Parametarski vs. Neparametarski modeli za klasifikaciju
- Neparametarski klasifikatori
 - K-najbližih susjeda (engl. K-nearest neighbors – KNN)
 - Stablo odlučivanja (engl. Decision tree)

Parametarski vs. Neparametarski modeli za klasifikaciju

Parametarski vs. Neparametarski modeli za klasifikaciju

- Algoritmi strojnog učenja mogu se grupirati u parametarske i neparametarske modele
- Upotrebom **PARAMETARSKIH** modela mi procjenjujemo parametre iz trening skupa podataka kako bismo naučili funkciju koja može klasificirati nove podatkovne primjere bez zahtjevanja originalnog trening skupa podataka
 - Tipičan primjer parametarskog modela je logistička regresija koju smo obradili ranije
- Nasuprot tome, **NEPARAMETARSKI** modeli ne mogu biti karakterizirani fiksnim skupom parametara, a broj parametara raste s brojem trening podataka
 - Tipični primjeri neparametarskih modela su klasifikator K-najbližih susjeda, stablo odlučivanja, kernel SVM

Klasifikator K-najbližih susjeda (engl. *K-nearest neighbors* – KNN)

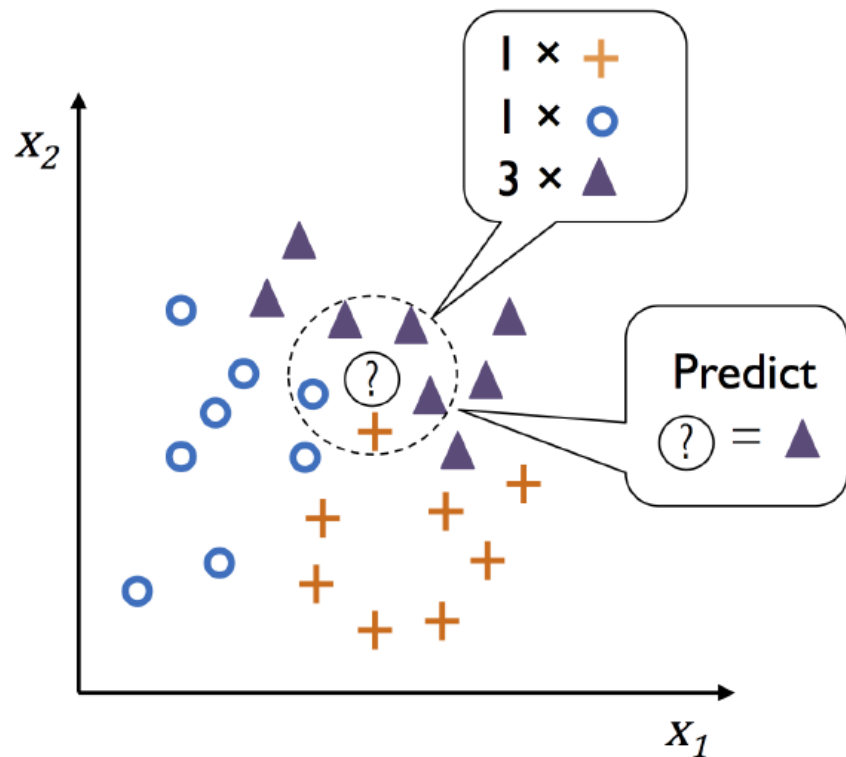
Klasifikator K-najbližih susjeda (KNN)

- Fundamentalno drugačiji od algoritama učenja o kojima smo dosad govorili ili ćemo govoriti na današnjem predavanju
- KNN je tipičan primjer metode lijenog učenja
- Naziva se "lijenom" ne zbog očigledne jednostavnosti, nego zato što ne uči diskriminirajuću funkciju iz trening podataka, već, umjesto toga, pamti trening skup podataka
- KNN pripada potkategoriji neparametarskih modela opisanih kao učenje zasnovano na instanci
- Modeli zasnovani na učenju na osnovu instance karakterizirani su pamćenjem skupa podataka za obuku, a metoda lijenog učenje je poseban slučaj učenja zasnovanog na instanci, koji je povezan s nula troškova tijekom procesa učenja

Klasifikator K-najbližih susjeda (KNN)

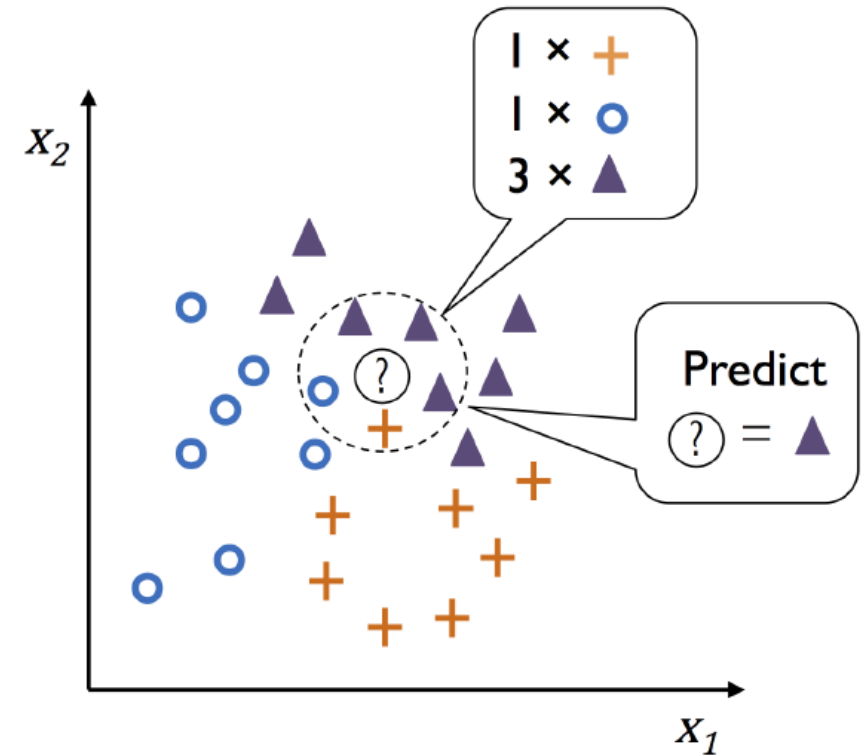
- Sam KNN algoritam je prilično jednostavan i može se rezimirati sljedećim koracima:

- 1. Odaberite broj K i metriku udaljenosti (rastojanja)**
- 2. Pronađite K-najbližih susjeda podatka koji želite klasificirati**
- 3. Dodijelite oznaku klase većinom glasova**



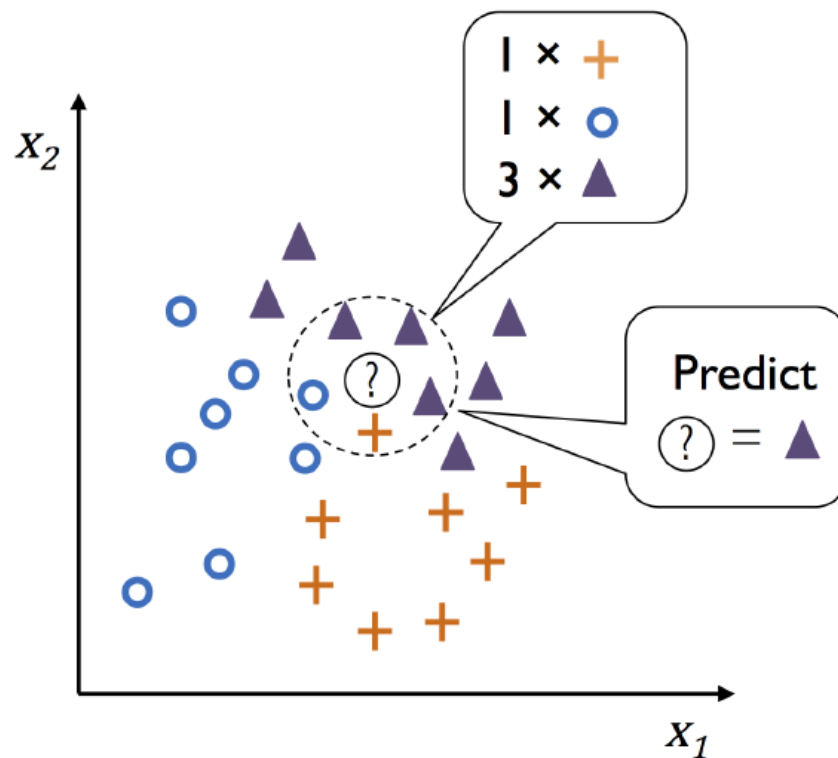
Klasifikator K-najbližih susjeda (KNN)

- Npr. novom podatkovnom primjeru („?” na slici) dodjeljuje se oznaka klase „trokuta” na osnovu većine glasova među njezinih pet najbližih susjeda
- Na osnovu odabrane metrike udaljenosti (rastojanja), KNN algoritam pronalazi K primjera u trening skupu koji su najbliži (najsličniji) podatku kojeg želimo klasificirati
- Oznaka klase podatka određuje se većinom glasova između tih K najbližih susjeda



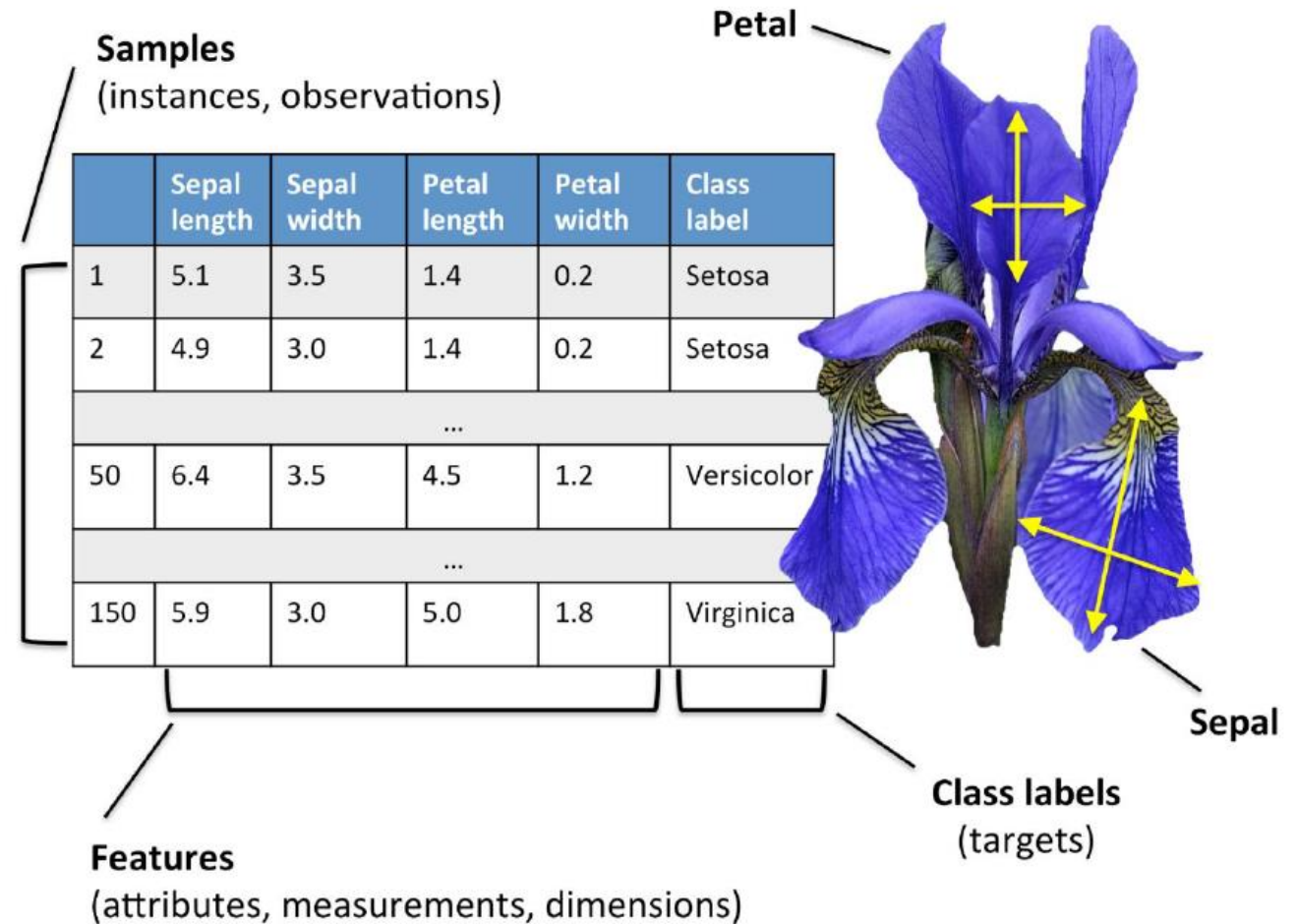
Klasifikator K-najbližih susjeda (KNN)

- Glavna **prednost** takvog pristupa zasnovanog na memoriji je što se klasifikator odmah prilagođava kako prikupljamo nove trening podatke
- Međutim, **nedostatak** je to što računalna složenost za klasifikaciju novih primjera raste s brojem primjera u trening skupu podataka
- Nadalje, ne možemo odbaciti trening podatke jer nije uključen nikakav korak učenja
 - Prostor za pohranu može postati izazov ako radimo s velikim skupovima podataka



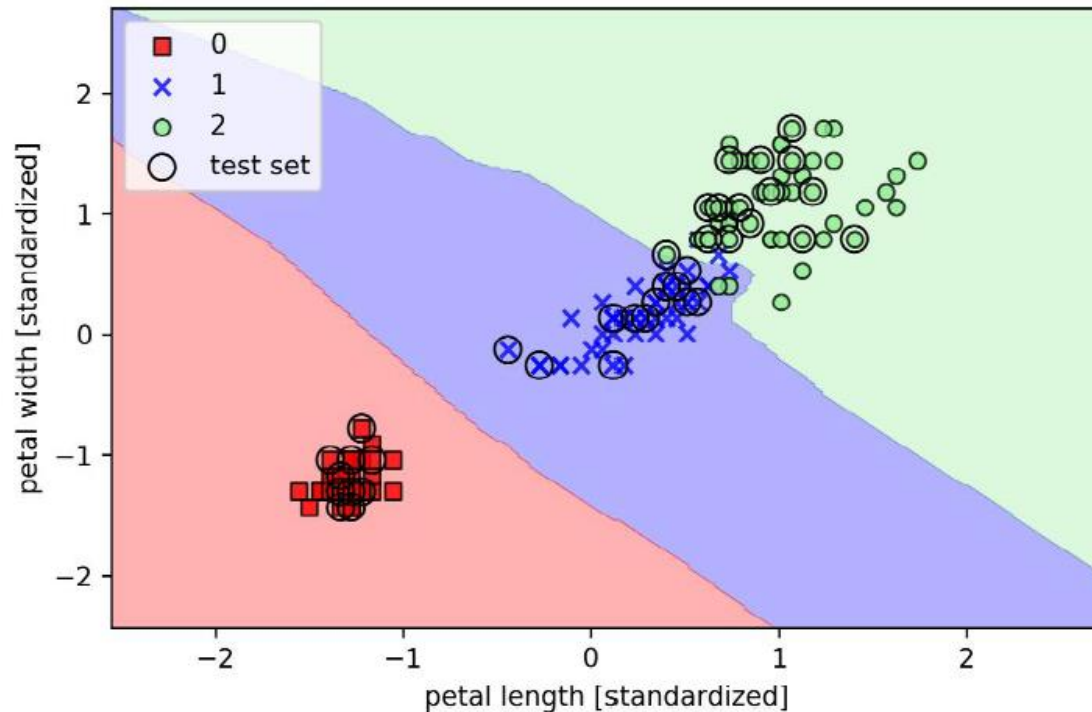
Klasifikator K-najbližih susjeda (KNN)

- Iris skup podataka
- Sadrži mjere 150 cvjetova perunike iz triju različitih vrsta – **Setosa, Versicolor i Virginica**
- Svaki primjer cvijeta predstavlja jedan redak u skupu podataka, a mjere cvijeta u centimetrima pohranjuju se kao stupci, koji se također nazivaju značajkama (atributima) skupa podataka



Klasifikator K-najbližih susjeda (KNN)

- Primjer primjene KNN na Iris supu odataka s tri klase cvjetova: Setosa, Versicolor, Virginica
 - KNN uz $K=5$ → relativno glatka granica odluke



- Kada jedna klasa ne prevladava (npr. uz $K=5$ rezultat glasanja je 2/2/1), u slučaju *scikit-learn* implementacija KNN algoritma će preferirati susjede s najbližim rastojanjem do podatka koji se klasificira
- Ukoliko susjedi imaju ista rastojanja, algoritam će izabrati oznaku klase koja je prva u trening skupu podataka

Klasifikator K-najbližih susjeda (KNN)

- Pravi izbor K ključan je za pronalaženje dobre ravnoteže između preprilagođavanja (*overfitting*) i nedovoljnog prilagođavanja (*underfitting*)
- Također, moramo se uvjeriti da smo odabrali metriku udaljenosti (rastojanja) koja je prikladna za značajke u skupu podataka
- Često se jednostavna Euklidska mjera udaljenosti koristi za primjere realnih vrijednosti, kao što su cvjetovi u Iris skupu podataka, koji imaju značajke mjerene u centimetrima
- Međutim, ako koristimo Euklidsku mjeru udaljenosti, važno je skalirati podatke da bi svaka značajka podjednako doprinijela udaljenosti

Klasifikator K-najbližih susjeda (KNN)

- **Minkowski udaljenost** samo je generalizacija Euklidske i Manhattan udaljenosti, koja se može napisati na sljedeći način

$$d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sqrt[p]{\sum_k |x_k^{(i)} - x_k^{(j)}|^p},$$

- $p=2 \rightarrow$ Euklidska udaljenost
 - $p=1 \rightarrow$ Manhattan udaljenost
- Mnoge druge metrike udaljenosti dostupne su u *scikit-learn* biblioteci i mogu biti predane kao *metric* parametar

Klasifikator K-najbližih susjeda (KNN)

- **„Prokletstvo dimenzionalnosti”**
- Važno je spomenuti da je KNN vrlo podložan preprilagođavanju (overfitting) zbog prokletstva dimenzionalnosti
- Prokletstvo dimenzionalnosti opisuje pojavu u kojoj prostor značajki postaje sve manji za sve veći broj dimenzija trening skupa podataka fiksne veličine
- Možemo čak zamisliti najbliže susjede kao da su predaleko u visokodimenzionalnom prostoru da bi dali dobru procjenu
- **Kako bismo pomogli u izbjegavanju prokletstva dimenzionalnosti, možemo koristiti odabir značajki i tehnike smanjivanja dimenzionalnosti podataka**
 - O tome ćemo razgovarati na jednom od sljedećih predavanja

Klasifikator K-najbližih susjeda (KNN) u *scikit-learn*

- KNN klasifikator je implementiran unutar sljedeće [klase](#):
- `class sklearn.neighbors.KNeighborsClassifier`
(`n_neighbors=5`, *, `weights='uniform'`, `algorithm='auto'`, `leaf_size=30`, `p=2`, `metric='minkowski'`, `metric_params=None`, `n_jobs=None`)
- Najvažniji parametri su:
 - `n_neighbors` – broj susjeda za korištenje prilikom klasifikacije (default je 5)
 - `p` – eksponent za Minkowski metriku (default je 2, što znači da se Euklidska udaljenost koristi kao mjera udaljenosti između dvaju uzoraka)

Klasifikator K-najbližih susjeda (KNN) - zaključak

- Faza učenja KNN algoritma je brza → **metoda lijenog učenja**
- **Samo jedan hiperparametar treba podesiti** (broj susjeda)
- Faza testiranja KNN algoritma može biti spora i treba (veliku) memoriju za pohranu cijelog skupa trening podataka
 - Problem s velikim skupovima podataka
- KNN algoritam **zahtjeva skaliranje podataka** jer koristi Euklidsku udaljenost između dvaju podataka za pronalaženje najbližih susjeda
 - Euklidska udaljenost je osjetljiva na amplitude
- **Pati od prokletstva dimenzionalnosti**
 - Performanse se mogu pogoršati kako se broj značajki povećava

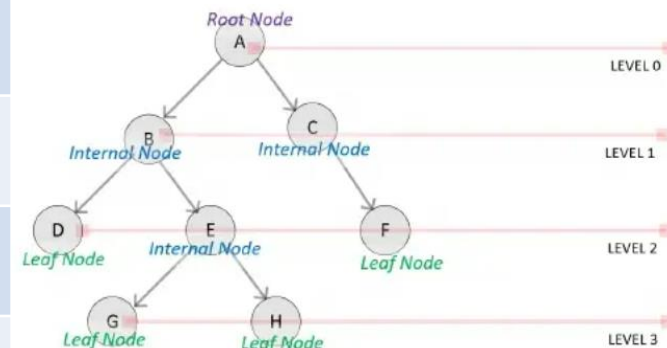
Klasifikator stabla odlučivanja (engl. Decision tree)

Što je stablo?

- U kontekstu algoritama, stablo je jedna od najkorisnijih podatkovnih struktura zbog svojih hijerarhijskih mogućnosti pohrane podataka
- Dok dizajniramo algoritme, koristimo stabla gdje god trebamo predstaviti hijerarhijske odnose među elementima podataka koje trebamo pohraniti ili obraditi
- Svako **stablo** ima konačan skup **čvorova** tako da ima početni podatkovni element koji se naziva **korijen** i skup čvorova spojenih vezama koje se nazivaju **grane**

Struktura stabla - Terminologija

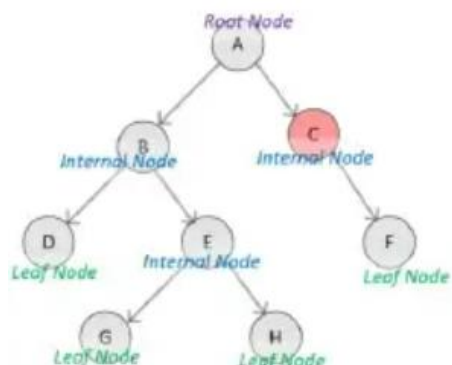
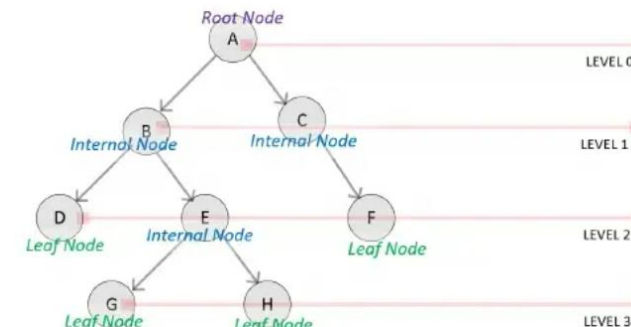
Izraz	Pojašnjenje
Korijenski čvor (Korijen)	Čvor bez roditelja naziva se korijenski čvor. U našem dijagramu, korijenski čvor je A . U algoritmima, obično, korijenski čvor ima najvažniju vrijednost u strukturi stabla.
Nivo (razina) čvora	Udaljenost od korijenskog čvora je razina čvora. U našem dijagramu, razina čvorova D , E i F je dva.
Bratski čvorovi	Dva čvora u stablu nazivaju se bratskima ako su na istoj razini. U našem dijagramu, čvorovi B i C su bratski čvorovi.
Roditeljski čvor i čvor potomak	Čvor F je potomak čvora C , ako su oni izravno povezani i razina čvora C je niža od razine čvora F . Pritom je čvor C roditelj čvora F .
Stupanj čvora	Stupanj čvora je broj potomaka koje čvor ima. Na primjer, u našem dijagramu, čvor B ima stupanj dva.
Stupanj stabla	Stupanj stabla jednak je maksimalnom stupnju koji se može naći među sastavnim čvorovima stabla. Na primjer, stablo prikazano na sljedećem dijagramu ima stupanj dva.
Podstablo	Podstablo stabla je dio stabla s odabranim čvorom kao korijenskim čvorom podstabla i svim potomcima kao čvorovima stabla. Na primjer, podstablo u čvoru E stabla predstavljenog u našem dijagramu sastoji se od čvora E kao korijenskog čvora i čvorova G i H kao dva potomka.
Čvor list	Čvor u stablu bez potomaka naziva se čvor list. Na primjer, na našem dijagramu D , G , H i F su četiri čvora lista.
Interni čvor	Svaki čvor koji nije ni korijenski ni listni čvor je interni (unutarnji) čvor. Interni čvor će imati najmanje jedan roditeljski čvor i najmanje jedan čvor potomak.



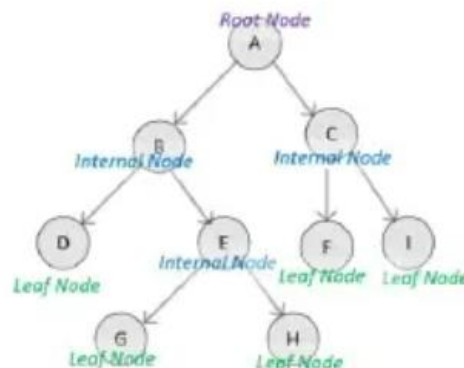
Stablo koje
ima četiri
nivoa
(razine) i 8
čvorova

Tipovi stabala?

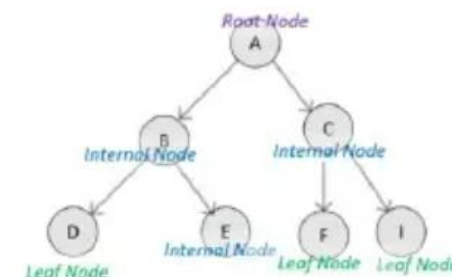
- **Binarno stablo** – ako je stupanj stabla dva, to se stablo naziva binarnim
- **Puno stablo** – ono u kojem su svi čvorovi istog stupnja, što će biti jednako stupnju stabla
- **Savršeno stablo** – posebna vrsta punog stabla u kojem su svi čvorovi listovi na istoj razini



Ne-puno, nesavršeno stablo



Puno, nesavršeno stablo



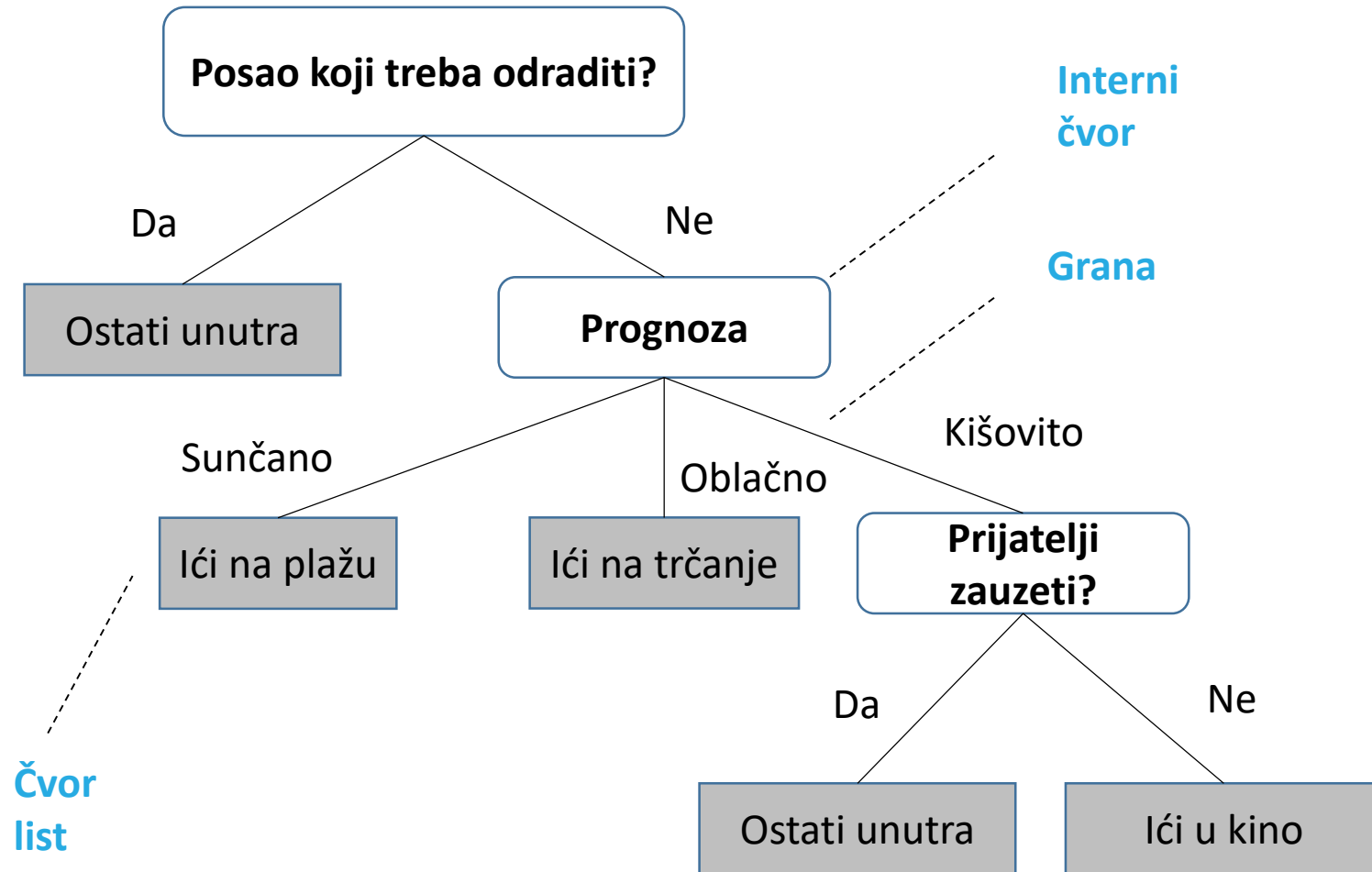
Puno, savršeno stablo

Stablo odlučivanja kao algoritam za klasifikaciju

- Stablo odlučivanja zasniva se na pristupu rekurzivnog dijeljenja, koji generira skup pravila koja se mogu koristiti za predviđanje labele
 - Ovaj model možemo zamisliti kao razlaganje naših podataka donošenjem odluke na temelju postavljanja niza pitanja (if/else)
- Počinje s korijenskim čvorom i dijeli se na više grana
- Interni čvorovi predstavljaju test u odnosu na određeni atribut (značajku), a rezultat testa je predstavljen grananjem na sljedeću razinu
- Stablo odlučivanja završava čvorovima listova koji sadrže odluke
- Proces se zaustavlja kada daljnja podjela više ne poboljšava ishod

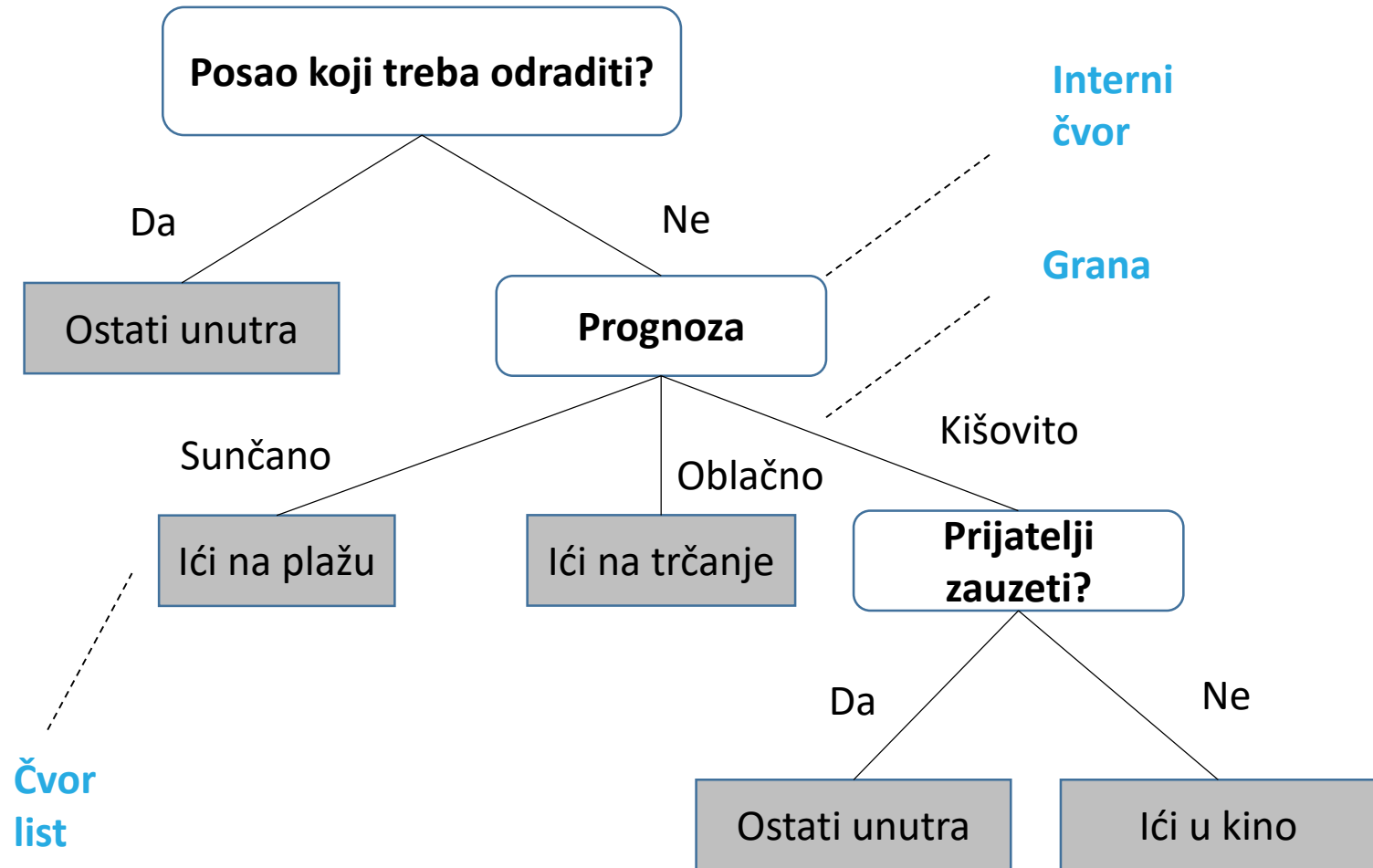
Stablo odlučivanja kao algoritam za klasifikaciju

- Jednostavan primjer u kojem koristimo stablo odlučivanja da odlučimo o aktivnosti na određeni dan
- Na temelju značajki u našem trening skupu podataka, model stabla odlučivanja uči niz pitanja kako bi zaključio oznake klasa trening primjera



Stablo odlučivanja kao algoritam za klasifikaciju

- Iako ova slika ilustrira koncept stabla odlučivanja zasnovanog na kategoričkim varijablama, isti koncept vrijedi ako su naše značajke realni brojevi, kao npr. u Iris skupu podataka
- Npr. mogli bismo jednostavno definirati graničnu vrijednost duž osi značajke *sepal width* i postaviti binarno pitanje:
- "Je li *sepal width* $\geq 2,8$ cm?"



Stablo odlučivanja kao algoritam za klasifikaciju

- Upotrebom algoritma odlučivanja započinjemo s korijenom stabla i razdvajamo podatke prema značajki koja dovodi do najvećeg informacijskog dobitka (engl. *information gain*) – za njegov izračun mogu se koristiti
 - Entropija, Gini nečistoća (engl. *Gini impurity*), Pogreška klasifikacije, ...
- U iterativnom procesu zatim možemo ponoviti ovaj postupak razdvajanja na svakom čvoru „potomku” dok čvorovi listovi ne budu čisti
- Što znači da su čvorovi listovi čisti?
 - To znači da svi trening primjeri u svakom čvoru pripadaju istoj klasi
 - Želimo li to?
- U praksi to može rezultirati vrlo dubokim stablom s mnogo čvorova, što može lako dovesti do overfittinga
 - Stoga obično želimo srezati stablo postavljanjem ograničenja za najveću dubinu stabla

Maksimizacija informacijskog dobitka (IG)

- Kako „dobiti maksimum za svoj novac”?
- Da bi razdvojili čvorove u najinformativnijim značajkama (atributima), trebamo definirati ciljnu funkciju koju želimo optimizirati pomoću algoritma učenja stabla
- Ovdje je naša ciljna funkcija da maksimiziramo informacijski dobitak (IG) u svakom razdvajanju, što ćemo definirati na sljedeći način

$$IG(D_p, f) = I(D_p) - \sum_{j=1}^m \frac{N_j}{N_p} I(D_j)$$

- f – atribut za izvršavanje razdvajanja,
- D_p – skup podataka „roditeljskog” čvora
- D_j – skup podataka j-tog čvora „potomka”
- I – mjera nečistoće
- N_p – ukupan broj trening primjera u „roditeljskom” čvoru
- N_j – ukupan broj primjera u j-tom čvoru „potomku”

Maksimizacija informacijskog dobitka (IG)

- Informacijski dobitak je jednostavno razlika između nečistoće „roditeljskog” čvora i zbroja nečistoća čvorova „potomaka”
- Što su niže nečistoće čvorova „potomaka”, veći je informacijski dobitak

$$IG(D_p, f) = I(D_p) - \sum_{j=1}^m \frac{N_j}{N_p} I(D_j)$$

- f – atribut za izvršavanje razdvajanja,
- D_p – skup podataka „roditeljskog” čvora
- D_j – skup podataka j-tog čvora „potomka”
- I – mjera nečistoće
- N_p – ukupan broj trening primjera u „roditeljskom” čvoru
- N_j – ukupan broj primjera u j-tom čvoru „potomku”

Maksimizacija informacijskog dobitka (IG)

- Međutim, zbog jednostavnosti i radi smanjivanja kombinatornog prostora pretrage, većina biblioteka (uključujući i *scikit-learn*) implementira binarna stabla odlučivanja
 - To znači da je svaki „roditeljski” čvor razdvojen na dva čvora „potomka”, D_{left} i D_{right}

$$IG(D_p, f) = I(D_p) - \sum_{j=1}^m \frac{N_j}{N_p} I(D_j)$$



$$IG(D_p, f) = I(D_p) - \frac{N_{left}}{N_p} I(D_{left}) - \frac{N_{right}}{N_p} I(D_{right})$$

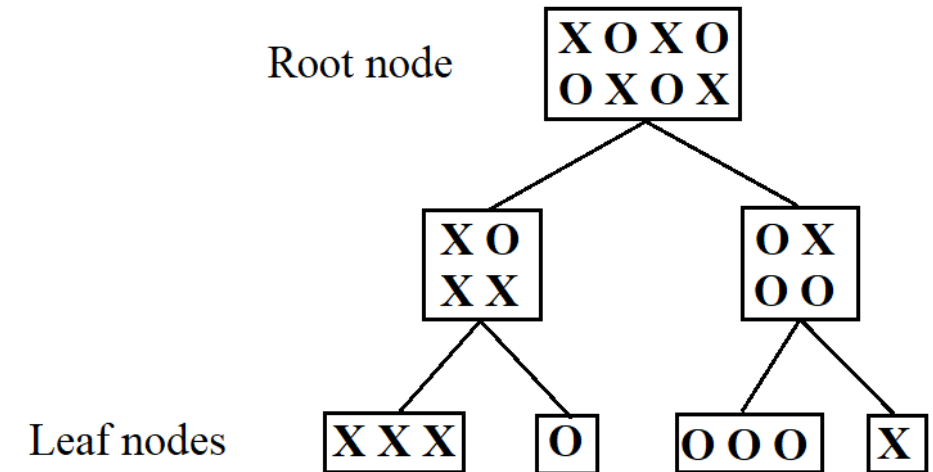
Maksimizacija informacijskog dobitka (IG)

$$IG(D_p, f) = I(D_p) - \frac{N_{left}}{N_p} I(D_{left}) - \frac{N_{right}}{N_p} I(D_{right})$$

- Kako mjeriti nečistoću /?
- Obično se u binarnim stablima odlučivanja koriste tri mjere nečistoće, tj. kriterija razdvajanja:
 - Entropija (I_H)
 - Gini nečistoća (I_G)
 - Pogreška klasifikacije (I_E)
- ...Ovdje nećemo ići u detalje kako se koja od njih računa...

Kako izraditi stablo odlučivanja?

- Jednostavan opći primjer za početak...
- Korijen sadrži skup kružića i križića
- Algoritam stvara kriterij koji pokušava razdvojiti kružiće od križića
- Na svakoj sljedećoj razini stablo odlučivanja stvara particije podataka za koje se očekuje da će biti sve homogenije i homogenije, počevši od razine 1 pa nadalje
- Savršeni klasifikator ima čvorove listove koji sadrže samo kružiće ili samo križiće
- Treniranje savršenih klasifikatora obično je teško zbog uvijek prisutne slučajnosti prilikom kreiranja trening skupa podataka



Kako izraditi stablo odlučivanja?

- Karakteristična značajka klasifikacije stablom odlučivanja je generiranje hijerarhije pravila koja se koriste za predviđanje klase tijekom izvođenja, a koja ljudi mogu protumačiti
 - Stvaranje ove hijerarhije pravila uključuje sljedeće korake
1. **Pronađite najvažniju značajku**: od svih značajki, algoritam identificira značajku koja najbolje razlikuje podatkovne primjere u trening skupu podataka s obzirom na njihovu labelu (klasu)
 - Izračun se zasniva na metrikama kao što su **informacijski dobitak (IG)**, **Gini nečistoća**, **entropija** i **pogreška klasifikacije**
 - **NAPOMENA**: ovo je parametar *criterion* u scikit-learn *DecisionTreeClassifier*

Kako izraditi stablo odlučivanja?

2. **Razdvajanje**: Koristeći najznačajniju identificiranu značajku, algoritam stvara kriterij koji se koristi za podjelu trening skupa podataka u dvije grane
 - Podatkovni primjeri koji zadovoljavaju kriterij
 - Podatkovni primjeri koji ne zadovoljavaju kriterij

3. **Provjerite čvorove listove**: Ako bilo koja rezultirajuća grana uglavnom sadrži oznake jedne klase, ta grana najčešće može postati konačna, što rezultira čvorom listom
 - Daljnje grananje bi moglo dovesti do *overfittinga*

Kako izraditi stablo odlučivanja?

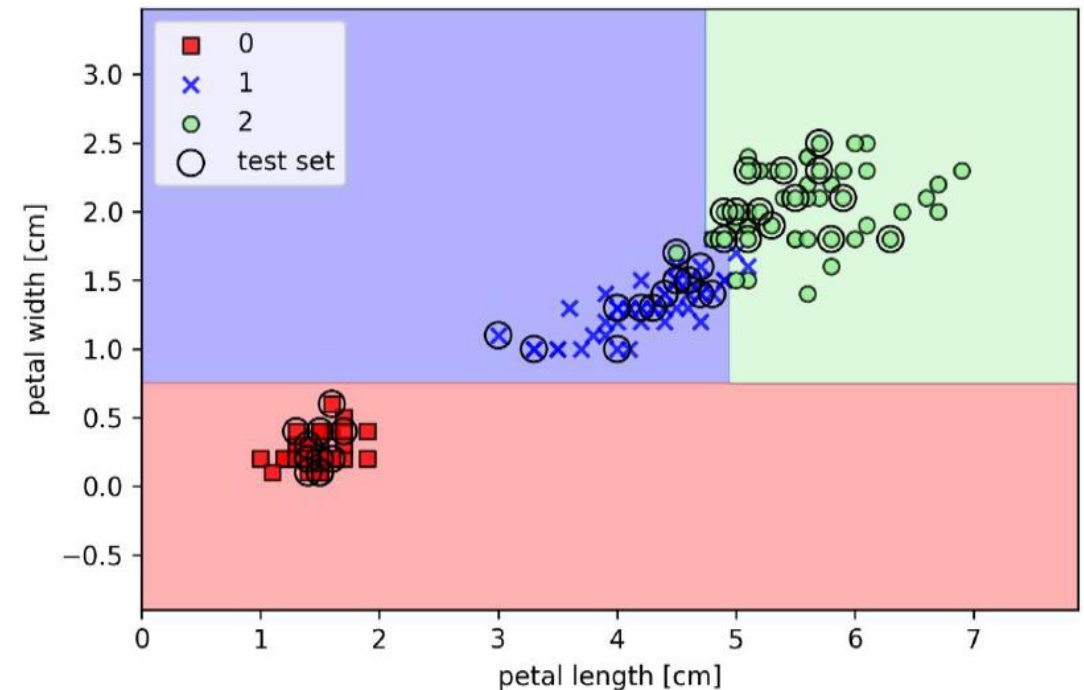
4. **Provjerite uvjete zaustavljanja i ponavljajte**: Ako zadani uvjeti zaustavljanja nisu ispunjeni, tada će se algoritam vratiti na korak 1 za sljedeću iteraciju. U suprotnom, model je označen kao obučen i svaki čvor rezultatnog stabla odlučivanja na najnižoj razini označen je kao čvor list
- Uvjet zaustavljanja može biti jednostavan poput definiranja broja iteracije (ponavljanja) ili se može koristiti zadani uvjet zaustavljanja, gdje se algoritam zaustavlja čim dosegne određenu razinu homogenosti za svaki od čvorova listova
 - Stabla odlučivanja mogu izgraditi složene granice odlučivanja dijeleći prostor značajki u pravokutnike
 - Međutim, moramo biti oprezni, jer što je stablo odlučivanja dublje, granica odluke postaje složenija, što lako može dovesti do *overfittinga*

Primjer izgradnje stabla odlučivanja

- Iris skup podataka → tri klase cjetova: Setosa, Versicolor, Virginica
- Cilj → trenirati stablo odlučivanja s maksimalnom dubinom 4, koristeći Gini indeks kao kriterij nečistoće
- **NAPOMENA:** Iako skaliranje značajki može biti potrebno za vizualizaciju, imajte na umu da skaliranje značajki nije potrebno za algoritme stabla odlučivanja

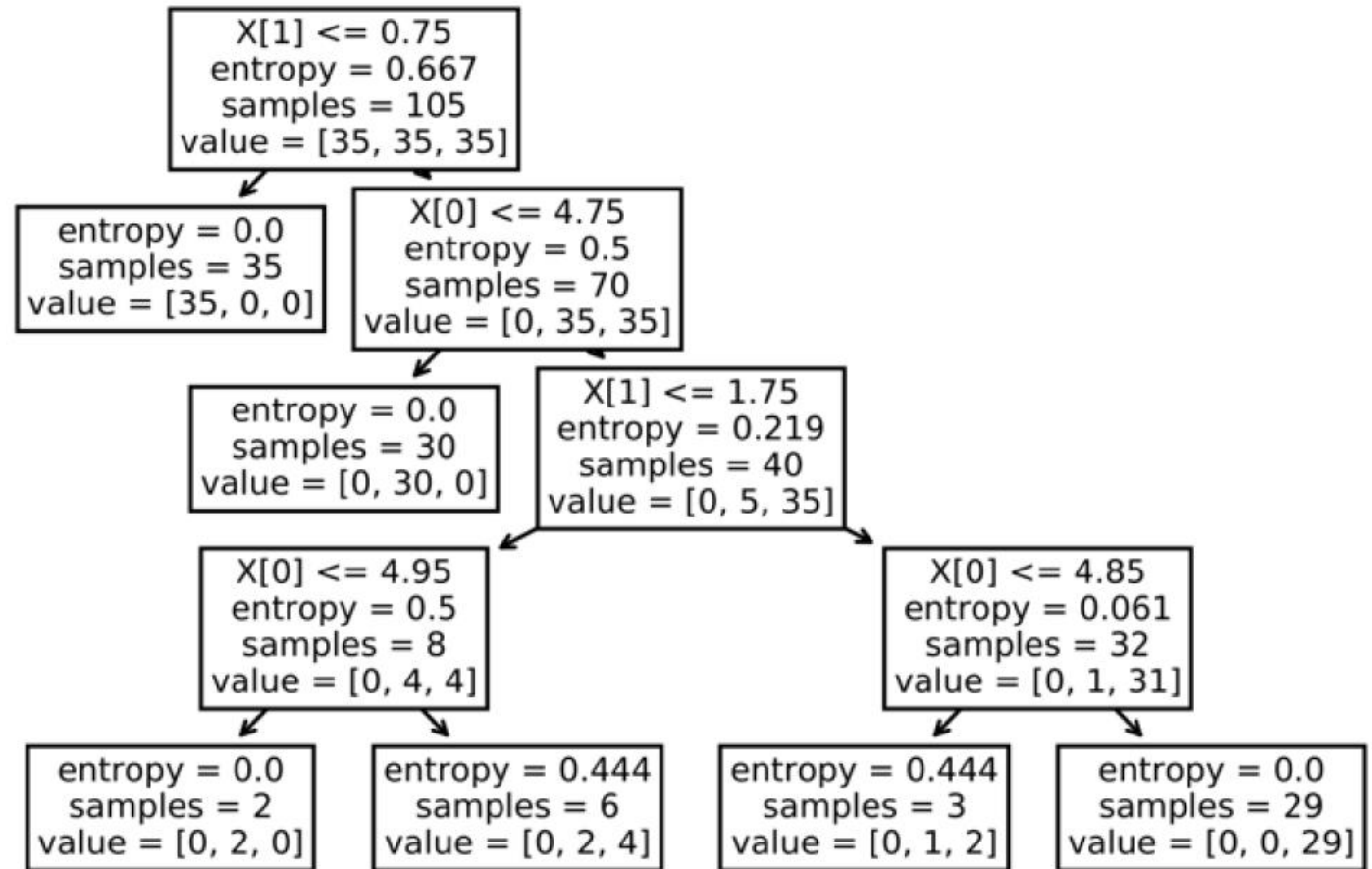
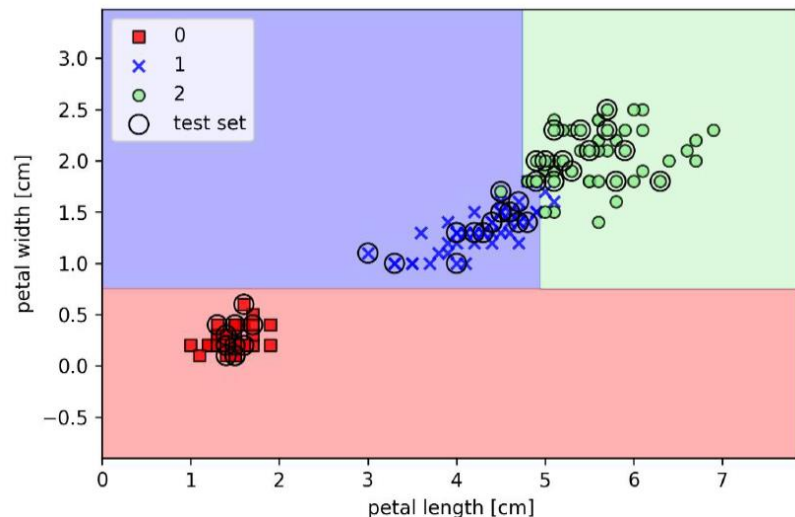
Primjer izgradnje stabla odlučivanja

- Iris skup podataka → tri klase cjetova: Setosa, Versicolor, Virginica
- Nakon provedbe treninga, dobit ćemo tipične granice odluke paralelne s osima za stablo odlučivanja



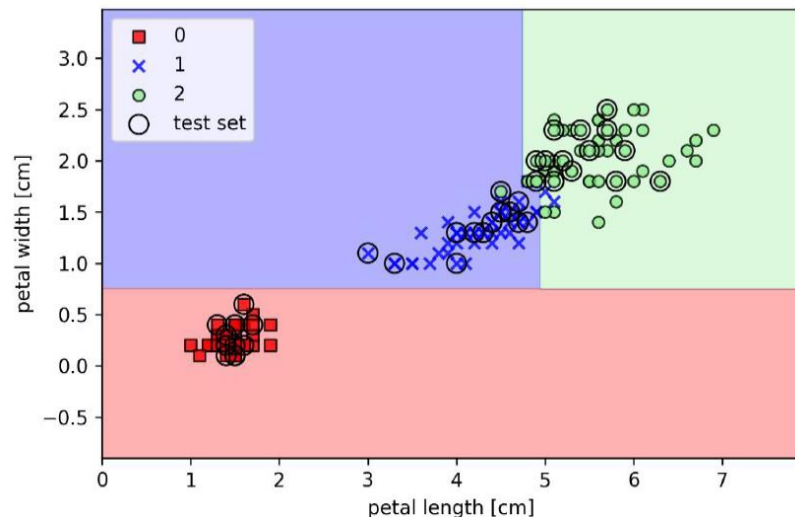
Primjer izgradnje stabla odlučivanja

- Postoje funkcije (*plot_three*) u *scikit-learn* biblioteci koje omogućavaju vizualizaciju modela stabla odlučivanja nakon treniranja



Primjer izgradnje stabla odlučivanja

- Moguće dobiti i ljepšu vizualizaciju upotrebom *Graphviz* programa kao *backenda* za prikaz *scikit-learn* stabla odlučivanja i biblioteke *PyDotPlus*

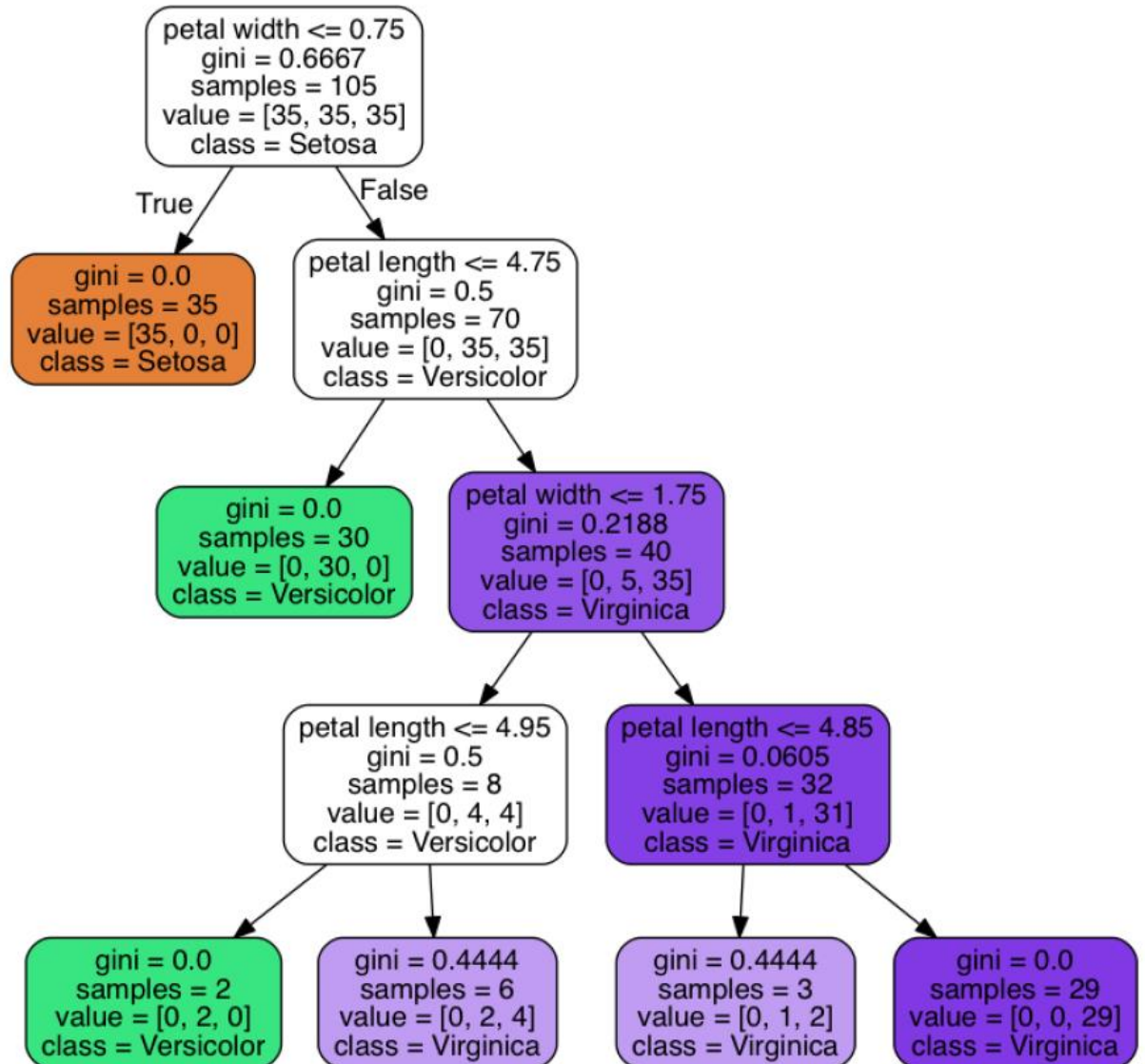


- Što bi značila ova svjetlija ljubičasta boja čvora lista?



Primjer izgradnje stabla odlučivanja

- Započeli smo primjer sa 105 trening podataka u korijenu i razdvojili smo ih u dva čvora potomka sa 35 i 70 podataka koristeći odsijecanje $\text{petal-width} \leq 0.75$ cm
- Nakon prvog razdvajanja, vidimo da je lijevi čvor potomak već čist i da sadrži samo primjere klase Setosa (Gini indeks=0)
- Dalja razdvajanja s desne strane su upotrijebljena za razdvajanje primjera iz klase Versicolor i Virginica



Stabla odlučivanja - prednosti

- Jednostavno za razumjeti i tumačiti – ljudi lako protumače modelom generirana pravila
- Modeli kao što je ovaj nazivaju se modeli „bijeke kutije” (engl. *Whitebox models*)
 - Nužni kad god je potrebna transparentnost za praćenje detalja i razloga donošenja odluke nekim modelom
 - Transparentnost ključna u aplikacijama u kojima želimo spriječiti pristranost i zaštititi ranjive zajednice
 - Npr. za kritične slučajeve uporabe u vladinim aplikacijama i osiguravateljskim industrijama
- Stabla se mogu vizualizirati
- Nema zahtjevne pripreme podataka
 - Druge tehnike često zahtijevaju normalizaciju podataka, potrebno je izraditi lažne varijable i ukloniti prazne vrijednosti
- Klasifikatori stabla odlučivanja dizajnirani su za izdvajanje informacija iz diskretnog problemskog prostora
 - To znači da su većina značajki kategoričke varijable, pa je korištenje stabla odlučivanja za treniranje modela dobar izbor

Stabla odlučivanja - nedostaci

- Moguće stvoriti presložena stabla koja ne generaliziraju dobro na novim podacima (*overfitting*) → treba ograničiti strukturu stabla odlučivanja (npr. postaviti maksimalnu dubinu stabla)
 - Ako stablo koje generira klasifikator stabla odlučivanja ide preduboko, pravila obuhvaćaju previše detalja, što rezultira modelom koji *overfitt-a*
 - Kad koristimo algoritam stabla odlučivanja, moramo biti svjesni da su stabla odlučivanja osjetljiva na *overfitting* i stoga moramo podrezati stablo, kad god je to potrebno, kako bismo to spriječili
- Nemogućnost generiranja/obuhvaćanja nelinearnih odnosa u pravilima koja stvara stablo odlučivanja
- Predviđanja stabala odlučivanja nisu ni glatka ni kontinuirana, već su aproksimacije konstantne po određenim dijelovima (nisu dobre za ekstrapolaciju)

Klasifikator stabla odlučivanja u *scikit-learn*

- Klasifikator stabla odlučivanja je implementiran unutar sljedeće [klase](#):
- `class sklearn.tree.DecisionTreeClassifier(*, criterion='gini', splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None, random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None, ccp_alpha=0.0)`
- Najvažniji parametri su:
 - `criterion` – funkcija za mjerenje kvalitete podjele/razdvajanja
 - `max_depth` – maksimalna dubina stabla
 - `min_samples_split` – min. broj uzoraka koje čvor mora imati prije dijeljenja (*default*=2)
 - `min_samples_leaf` – min. broj uzoraka koje čvor list mora posjedovati (*default*=1)
 - `max_leaf_nodes` – maksimalan broj čvorova listova koje stablo može imati (*default*=None)

P i t a n j a ?